Multi-Model Grokking Baseline (Equal-Compute, 5k Budget)

Yazan Kbaili, Ali Salloum — March 2026

Contributions

Yazan Kbaili: Theory and related work; equal-compute protocol; multi-model pipeline implementation; experiments and figures.

Ali Salloum: Experimental setup and hyperparameters; debugging and pipeline fixes; evaluation and plotting; report write-up and interpretation.

---

1. Goal

We want a solid baseline for **delayed generalization (grokking)** when using several models:

1. Train 4 specialist models on disjoint shards of the training set.
2. Merge them by averaging their weights.
3. Compare this merged model vs a random-init baseline, with the same step budget.

Question: does starting from averaged specialists help grokking (speed or final performance) compared to a single model trained from scratch, under a short 5k-step budget?

---

2. Background

2.1 Grokking

In **grokking** (Power et al., 2022), the model first **memorizes** the training set (high train accuracy, low val accuracy) and only later **generalizes** (val accuracy jumps). The gap can be huge in terms of steps.

Early on, the model fits training examples in a way that doesn't transfer (e.g. lookup-style). With enough training and strong regularization (e.g. weight decay), the optimizer can move toward simpler, general solutions (e.g. the real modular arithmetic rule). So you get a sharp "memorizing" minimum and a broader "generalizing" minimum; weight decay pushes toward the latter.

From a theoretical angle, the transition from memorization to generalization can be seen as a kind of **phase transition**: the model first settles into a solution that fits the training data but has high complexity (e.g. encoding individual examples), and only under sustained regularization does it shift to a simpler solution that generalizes. The loss landscape is often described as having a narrow memorizing basin and a wider generalizing basin; the dynamics depend on data size, model capacity, and the strength of weight

decay. This makes grokking a useful setting to study how different training schemes (e.g. multi-model then merge) affect when and whether that transition happens.

2.2 Why modular arithmetic?

Modular addition ($a + b \equiv c \pmod{p}$) with ($p = 97$) is a standard grokking setup: the rule is simple, the space is finite (9,409 equations), and with limited data and weight decay you get a clear memorize-then-grok curve. That makes it a good testbed for whether multi-model + merge changes when or how well grokking happens.

Theoretically, the task has a **unique correct rule** (the group operation mod (p)), so we can tell when the model has truly generalized rather than just memorized. Practically, we can evaluate on the full equation set and report exact accuracies, and the small scale (2-layer transformer, ~455K params) keeps runs fast and reproducible. The 50% train / 50% val split is common in the grokking literature so that the model must generalize beyond the training equations to do well on validation.

2.3 Weight averaging (model soup)

Averaging parameters of models trained on different subsets is a simple way to combine them. Each model specializes on its subset; the average can smooth the solution and sometimes generalize better. Here we check whether specialists that overfit their shards, when averaged, give the merged model a better starting point to grok on the full set.

Theory: In model-soup and federated-averaging settings, averaging can improve generalization when the individual models sit in compatible basins (e.g. same "generalizing" region). If specialists instead sit in **different** memorizing basins, their average can land in a worse place—e.g. between basins—and the merged model may need many more steps to escape. So the outcome depends on how aligned the specialist solutions are; in our setup we test the naive case (no coordination, no curriculum) to establish a lower bar.

We use **element-wise mean** of the transformer state dicts only. No learned weighting or distillation.

2.4 Regularization

Weight decay (AdamW, coefficient 1.0) shrinks weights toward zero and discourages heavy memorization, favoring flatter minima that tend to generalize. Same setup for specialists and for merged/baseline so the comparison is fair.

Without strong weight decay, models often stay in the memorization regime; with it, the delayed jump to high validation accuracy (grokking) can appear. The value 1.0 is in line with the original grokking paper and keeps the comparison consistent with prior work.

---

3. Method: multi-model protocol

Three phases: (1) train N specialists on N disjoint shards, (2) average their weights into one model, (3) compare merged model and a random-init baseline on the full data with the same step budget.

3.1 Phase 1 — Specialists

- **Data:** Full training set (4,704 equations) split into **N = 4** disjoint shards. Random permutation with seed 42, then split into 4 contiguous chunks. Each shard has 1,176 equations; no overlap.
- **Training:** One transformer per shard. Specialist (i) sees only shard (i). Shared validation set (4,705 equations) for all.
- **Steps:** Equal compute: `specialist_steps = final_steps // N` → 5,000 steps per specialist. No early stopping; each runs the full 5k steps.
- **Output:** N state dicts.

3.2 Phase 2 — Merge

- Average the N transformer state dicts element-wise. Optimizer state is dropped. Result saved as `merged_weights.pt`. The merged model is at the centroid of the N solutions in parameter space; the idea is it might keep shared structure and smooth out shard-specific noise.

3.3 Phase 3 — Merged vs baseline

- **Merged:** Same architecture, load merged weights, train on **full** 4,704 equations for 5,000 steps.
- **Baseline:** Same architecture, **random** init, same full data, 5,000 steps. Same optimizer, LR, weight decay, everything.
- We compare validation accuracy and learning curves.

3.4 Summary

Phase	What happens	Output
1	Train 4 specialists on 4 disjoint shards (5k steps each)	4 state dicts
2	Element-wise mean of the 4 state dicts	1 merged state dict
3	Train baseline from scratch (5k steps)	Merged and baseline metrics

---

4. Setup

4.1 Model and optimizer

- **Architecture:** Decoder-only transformer. 2 layers, 4 heads, d_model=128, max context 50. ReLU, no dropout. ~455K params.
- **Optimizer:** AdamW-style (project variant). LR 1e-3, betas (0.9, 0.98), weight decay 1.0 (to_zero), 10-step linear warmup then constant LR.
- **Device:** Single GPU (gpu=0).

Practical notes: We keep the architecture small so that specialist training and merged/baseline runs finish in a few hours on one GPU. No dropout so that the main regularizer is weight decay; adding dropout

would change the grokking dynamics. The learning rate and weight decay match the original grokking setup where possible so that our baseline is comparable to published results.

4.2 Data and task

- **Task:** Next-token prediction for modular addition. Sequences like `<EOS> a + b = c <EOS>`; labels are tokens after `=`. Vocab size 97.
- **Splits:** 50% train → 4,704 equations; 4,705 validation. Batch size from dataset (e.g. min(512, ceil(train_size/2))). Validation in one batch when possible.

Protocol recap: 4 specialists, 4 shards (seed 42), 5k steps each, then merge by averaging. Merged and baseline both get 5k steps on full data.

Why 4 specialists and 5k steps: We use N=4 to get a non-trivial split (each specialist sees 1/4 of the data) without blowing up compute. The 5k-step budget for merged and baseline is short enough that the baseline can still grok (we see it reach 100% val by ~2.9k steps) but tight enough to distinguish a good vs bad initialization. Equal compute means total specialist steps = one "equivalent" full run (20k total, 5k per specialist), so we're not giving the multi-model side more steps than the baseline.

4.3 Training and logging

- PyTorch Lightning. Max steps only; no epoch cap.
- Validation every 25 epochs. Metrics: train loss, full-train accuracy, val loss, val accuracy. CSVLogger per run.

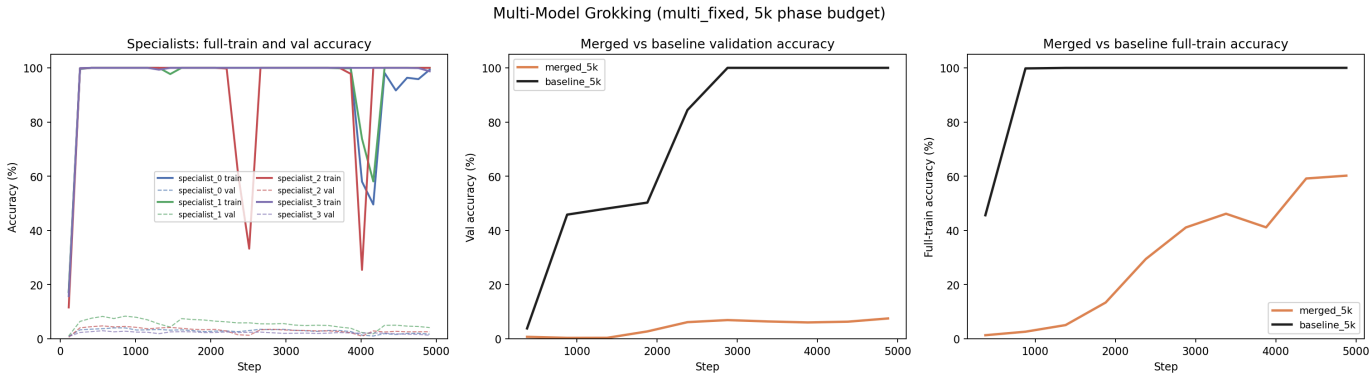
4.4 Fixes we had to make

1. **Shards:** Lightning's second `prepare_data()` was overwriting our per-specialist datasets. We apply the custom train/val datasets in `setup()` so each specialist only sees its shard.
2. **Accuracy scale:** Accuracy is in percent [0, 100]. We use threshold 99.0 for 99%, not 0.99.
3. **Validation frequency:** Validating every batch was too slow. We use `check_val_every_n_epoch=25` instead.

Reproducing: Run from the repo root so paths to `data/` and `logs/` resolve correctly. If you change the train/val split or seed, specialist shards will differ and numbers will not match exactly. Logs go to `logs/<experiment_name>/`; use the same `--experiment_name` for train_multi and run_merged_baseline so that plot scripts find the right metrics.

---

5. Results



5.1 Specialists (5k steps each, 1,176 eq/shard)

Model	Step when full-train = 100%	Peak val acc (%)	Step @ peak val	Final val acc (%)
specialist_0	413	4.017	713	1.722
specialist_1	413	8.353	863	4.145
specialist_2	413	4.761	563	2.593
specialist_3	413	2.976	563	1.296

All hit 100% training accuracy on their shard by step 413. Validation stays low (1–8%), which fits with overfitting to the shard.

5.2 Merged vs baseline (5k steps, full 4,704 train eq)

Model	Max logged step	Peak val acc (%)	Step @ peak val	Final val acc (%)	Final full-train acc (%)
merged_5k	4879	7.503	4879	7.503	60.204
baseline_5k	4879	100.000	2879	100.000	100.000

Thresholds:

Metric	merged_5k	baseline_5k
First step with val ⩾ 50%	not reached	1879
First step with val ⩾ 90%	not reached	2879
First step with val ⩾ 99%	not reached	2879
First step full-train ⩾ 50%	4379	879
First step full-train ⩾ 100%	not reached	1879

6. Interpretation

With a 5k-step budget, the merged model (from naive specialist averaging) does **not** beat the baseline. The baseline reaches 100% validation accuracy; the merged model stays around 7.5%.

Specialists overfit to their shards. Averaging their weights doesn’t put the merged model in a basin that generalizes within 5k steps. The baseline, trained from scratch on the full 50% training set, sees the whole distribution and can find the modular-addition rule. The merged model starts from a mix of four shard-specific solutions that can conflict or cancel; in this setup that initialization doesn’t help.

So this is a **negative baseline**: naive averaging isn’t enough here. Any better method (smarter merge, curriculum, distillation, etc.) should do better than this.

---

7. Reproducibility

From the repo root:

- Multi-model pipeline: `python scripts/train_multi.py --equal_compute --train_data_pct 50 --final_steps 20000` (or `scripts/run_multi_fixed.ps1`). This gives specialist checkpoints and `merged_weights.pt`.
- Merged and baseline 5k: `python scripts/run_merged_baseline.py --final_steps 5000`.
- Curves: `scripts/plot_multi_fixed_5k.py` on the logged metrics.

Exact commands and paths are in the README.

---

8. Conclusion

For 50% training data and 5k steps for merged/baseline:

- **baseline_5k:** 100% final validation accuracy.
- **merged_5k:** 7.5% final validation accuracy.

The random-init baseline clearly beats merged specialist averaging in this setting. Future work should be compared against this baseline.

9. Next Steps

- **No Regularization:** try to train the same step without Regularization to delay grokking
- **Less Data:** Use only 5% of the data for training instead of 50% but with more epochs
- **Fine-tuning for Merged Model**